

HCSS DATALAB ASSIGNMENT

Dutch Parliament

Energy Security Framing

Technical Documentation & Project Report

ParlaMint-NL 2019–2022 · Framing Analysis Tool · May 2025

9,102

Utterances

337

Speakers

1,127

Sessions

2019–2022

Period

Research Question

When and how do Dutch parliamentarians shift from treating energy as a technical or economic issue to treating it as a strategic security vulnerability? And does this shift correlate with geopolitical events such as Russia's invasion of Ukraine in February 2022?

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. System Architecture	5
3. Data Processing — process_parlamint.py.....	6
3.1 Input Format	6
3.2 Text Normalisation	6
3.3 Keyword Dictionaries	6
3.4 Scoring Logic	7
3.5 Derived Metrics	7
4. Speaker Enrichment — enrich_speakers.py	9
4.1 TEI/XML Parsing	9
4.2 Party Extraction Strategy	9
4.3 Speaker Aggregates	9
5. Advanced Statistical Analysis.....	10
5.1 Analytical Modules	10
5.2 Why Mann-Whitney U, Not a T-Test?.....	10
5.3 Securitization Index	11
6. Dashboard — app.py	12
6.1 Architecture	12
6.2 Dashboard Sections	12
6.3 Hover Label Fix	12
6.4 AI Summarization Component	13
7. Database Schema Design	14
7.1 Schema Pattern: Star Schema	14
7.2 The Bridge Table — Slowly Changing Dimension Type 2.....	14
7.3 Pre-Computed Scores vs. On-Demand Calculation	14
7.4 Index Strategy.....	15
8. Analytical Findings.....	16
8.1 Securitization Trajectory (2019–2022).....	16
8.2 Frame Co-occurrence.....	16
8.3 Actor-Level Patterns	16
8.4 The Critical Materials Gap.....	17
9. Technology Stack	18
10. Limitations & Future Work.....	19

10.1 Current Limitations.....	19
10.2 Future Improvements	19
11. Setup & Usage	20
11.1 Installation	20
11.2 Pipeline Execution	20
11.3 File Structure.....	21

1. Executive Summary

This report documents the design, implementation, and analytical findings of the **Dutch Parliament Energy Security Framing Tool** — a full-stack data analysis application built for the HCSS Datalab assignment. The tool processes the ParlaMint-NL parliamentary corpus (2019–2022), applies a keyword-based framing classification system, and presents results through an interactive Streamlit dashboard with AI-powered analysis capabilities.

Key Findings

- 9,102 energy-relevant utterances were extracted from 1,127 parliamentary sessions across 337 unique speakers (2019–2022).
- Security focus density increased by 227.3% from the pre-2022 average (2.12 hits/1,000 words) to the 2022 average (6.94 hits/1,000 words), with the sharpest inflection in February–March 2022 following Russia's invasion of Ukraine.
- 16.8% of energy utterances simultaneously contain security framing (the 'energy × security overlap'), rising sharply in Q1 2022.
- Dominant framing is 'energy-general' (69.8%), followed by economic (10.8%), climate (9.3%), security (7.8%), and critical materials (2.3%).
- Mark Rutte (87 utterances), Rob Jetten (46), and Sjoerd Sjoerdsma (40) are the highest-volume energy-security speakers.
- Critical minerals remain dramatically under-discussed (2.3% primary framing), suggesting a strategic blind spot in Dutch parliamentary discourse.

2. System Architecture

The tool is organised as a five-stage pipeline, each stage implemented as a separate Python module. Stages are loosely coupled through CSV files, making it easy to re-run or replace individual components without reprocessing the entire corpus.

Stage	Script	Input	Output
1. Ingest & Score	process_parlamint.py	ParlaMint-NL .txt/.tsv files	energy_security_utterances.csv
2. Enrich Speakers	enrich_speakers.py	CSV + ParlaMint TEI/XML	energy_security_utterances_enriched.csv
3. Keyword Audit	keyword_audit.py	Enriched CSV	keyword_audit/ CSVs for manual review
4. Advanced Analysis	advanced_analysis_energy_security.py	Enriched CSV	analysis_outputs/ (22 CSVs + plots)
5. Dashboard	app.py	Enriched CSV	Interactive Streamlit web application

Design Principle

Each stage writes a self-contained output file. A researcher can stop after stage 1, inspect the CSV in Excel, improve keyword dictionaries using keyword_audit.py, re-run stage 1, and then re-run stages 2–5 without losing any work. The pipeline is stateless and fully reproducible.

3. Data Processing — process_parlamint.py

It reads every .txt and .tsv file in the ParlaMint-NL English text corpus, applies text normalisation, matches keywords, and outputs a flat CSV containing one row per energy-relevant utterance.

3.1 Input Format

ParlaMint distributes parliamentary text in two formats: TEI/XML (rich metadata) and plain .txt/.tsv (text content). This script consumes the plain text files, which follow a simple two-column tab-separated format:

```
# Format: <utterance_id> <raw_text>
ParlaMint-NL_2022-03-10-tweedekamer-4.u15  Mr. President, the energy
                                         situation in Europe has
                                         fundamentally changed...
```

3.2 Text Normalisation

Raw parliamentary text is messy — it contains titles, procedural phrases, speaker references, and transcription artefacts. Normalisation makes keyword matching reliable across different speech styles:

```
def normalize_text(raw_text: str) -> str:
    text = raw_text.lower()
    text = re.sub(r'[^a-z0-9\s\-\,]', '', text) # remove punctuation
    tokens = nltk.word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    if t not in STOPWORDS and len(t) >= 3] # lemmatize & filter
    return ''.join(tokens)
```

Lemmatisation (e.g. 'dependencies' → 'dependency', 'invaded' → 'invade') ensures that keyword patterns catch inflected forms without needing to list every variant explicitly. Stopword removal eliminates high-frequency function words that would otherwise inflate word counts and dilute density scores.

3.3 Keyword Dictionaries

Five keyword dictionaries define the framing dimensions. Each dictionary was built iteratively: an initial list of obvious terms was compiled, then expanded using the keyword_audit.py tool, which surfaces the most frequent words and phrases in the corpus that are NOT yet matched by any dictionary.

Dimension	Core Terms (examples)	Size
Energy	energy, gas, lng, pipeline, nord stream, groningen, hydrogen, energy transition, gasunie	~65 terms

Security	national security, russia, ukraine, nato, sanctions, strategic autonomy, dependency, critical infrastructure	~75 terms
Economic	energy price, inflation, purchasing power, supply chain, trade balance, subsidy, competitiveness	~50 terms
Climate	climate change, co2, emissions, paris agreement, green deal, net zero, carbon tax	~45 terms
Critical Materials	critical minerals, rare earths, lithium, cobalt, semiconductor, battery materials	~40 terms

Why phrase-aware matching matters

Simple word matching would count 'energy' in 'energy drinks' or 'Russia' in a general foreign policy discussion unrelated to energy. The script pre-compiles all keyword patterns using `re.compile()` with word-boundary anchors (`\b`), and phrases are matched before single words. This prevents a phrase like 'nord stream' from being double-counted as both 'nord' and 'stream'.

3.4 Scoring Logic

For each utterance, five raw scores are computed (one per dimension). Scores represent the count of keyword matches in the normalised text — a single utterance can score 5 on security if it mentions Russia, sanctions, NATO, dependency, and strategic vulnerability in the same speech.

```
energy_score = count_patterns(normalized_text, ENERGY_PATTERNS)
security_score = count_patterns(normalized_text, SECURITY_PATTERNS)
# ... etc for economic, climate, minerals
```

```
# Only keep utterances that actually mention energy or minerals:
if energy_score == 0 and minerals_score == 0:
    continue # skip — not about energy
```

3.5 Derived Metrics

Metric	Formula	Purpose
Keyword Density	$\text{score} / \text{word_count} \times 1000$	Length-normalised intensity. Enables fair comparison between a 30-word interjection and a 500-word speech.
Primary Framing	<code>argmax(security, economic, climate, minerals, energy-general)</code>	Single dominant frame label for filtering and visualisation.
Securitization Index	$\text{security_score} / (\text{economic} + \text{climate} + \text{minerals} + 1)$	Ratio of security to all other non-energy frames. Value > 1 means security language dominates.

Example Calculation

Utterance: 'Russia's invasion has made our gas dependency a direct national security threat. We must diversify energy supply away from Russian pipelines.' Scores: energy=4 (gas, dependency, energy supply, pipelines), security=4 (Russia's invasion, dependency, national security threat, Russian). Word count: 32. Security density = $4/32 \times 1000 = 125.0$. Securitization Index = $4/(0+0+0+1) = 4.0$. Primary framing = security. is_energy_security = True.

4. Speaker Enrichment — `enrich_speakers.py`

The plain .txt files contain utterance IDs and text but no speaker metadata. Speaker names, party affiliations, gender, and role are stored in the ParlaMint TEI/XML files. This script parses those XML files and joins speaker metadata onto the utterance CSV.

4.1 TEI/XML Parsing

ParlaMint XML files follow the TEI (Text Encoding Initiative) standard. The relevant structure looks like this:

```
<person xml:id='MarkRutte'>
  <persName>Mark Rutte</persName>
  <sex value='M' />
  <affiliation ref='#VVD' />
</person>
...
<u xml:id='ParlaMint-NL_2022-02-28.u12' who='#MarkRutte'>
  <seg>We cannot continue to depend on Russian gas.</seg>
</u>
```

4.2 Party Extraction Strategy

TEI affiliation data is present but inconsistently structured. The script implements a four-level fallback strategy:

- Parse affiliation/@ref and look it up in the <org> dictionary within the same file.
- If the org lookup fails, clean the raw ref attribute (e.g. '#party.VVD' → 'VVD').
- Fall back to the text content of <affiliation//orgName>.
- If all TEI extractions fail, apply a hardcoded speaker-name → party mapping for the 30 most-active speakers, built from public records.

Known Limitation

The TEI party extraction currently leaves approximately 80% of utterances with 'Unknown' party affiliation, because the ParlaMint-NL TEI files store party data in a root corpus file rather than in individual session files. The name-based fallback mapping partially compensates, correctly assigning parties to the 30 highest-volume speakers who account for a disproportionate share of security-framed utterances.

4.3 Speaker Aggregates

After enrichment, the script appends speaker-level aggregate columns to every row: total utterances per speaker, average security density per speaker, and energy-security overlap count. These pre-computed aggregates make the dashboard speaker-ranking charts fast even without a database.

5. Advanced Statistical Analysis

This standalone script performs eight analytical modules on the enriched CSV, writing results to a structured output directory. It is decoupled from the dashboard so it can be run as a batch job overnight on the full corpus.

5.1 Analytical Modules

Module	Method	Output Files
Descriptive Statistics	Pandas aggregations: mean, std, percentiles per framing dimension and year	01_overview.csv, 02_primary_framing_distribution.csv
Frequency & Co-occurrence	Jaccard similarity between frame pairs; heatmap of co-activation rates	04_frame_cooccurrence_matrix.csv, 05_frame_cooccurrence.csv
Trend Analysis	Monthly and yearly rolling means of density scores; % of debate share	06_yearly_trends.csv, 09_monthly_trends.csv
Inferential Tests	Mann-Whitney U test (pre/post-2022) for each density metric; non-parametric for skewed distributions	11_pre_post_statistical_tests.csv
Framing Distribution Test	Chi-square test of independence: period × primary_framing contingency table	13_chi_square_period_framing.csv
Correlation Analysis	Spearman rank correlations between all density pairs; corrects for outliers	14_spearman_correlation_matrix.csv
Speech Clustering	K-Means on TF-IDF-style density vectors; PCA to 2D for visualisation; silhouette scoring to choose k	20_speech_cluster_profiles.csv
Speaker Clustering	K-Means on per-speaker aggregate density vectors; profiles each cluster	22_speaker_cluster_profiles.csv

5.2 Why Mann-Whitney U, Not a T-Test?

Statistical Note

Security density scores are heavily right-skewed: the median is 0.0 (most utterances contain no security keywords) while the mean is 3.36 and the maximum is 166.67. Parametric tests like the t-test assume approximately normal distributions and would produce unreliable p-values. The Mann-Whitney U test is a non-parametric alternative that compares the rank-order distributions

of two groups, making it robust to extreme outliers. The test asks: 'Is a randomly drawn utterance from 2022 more likely to have a higher security density than a randomly drawn utterance from 2019–2021?'

5.3 Securitization Index

$\text{securitization_index} = \text{security_score} / (\text{economic_score} + \text{climate_score} + \text{minerals_score} + 1)$

Examples:

Pure security speech (no economic/climate): $8 / (0 + 0 + 0 + 1) = 8.0 \rightarrow \text{high}$

Balanced security + economic: $4 / (4 + 0 + 0 + 1) = 0.8 \rightarrow \text{medium}$

Economic-dominated, some security: $2 / (6 + 2 + 0 + 1) = 0.22 \rightarrow \text{low}$

6. Dashboard — app.py

The dashboard is built with Streamlit and Plotly. It is designed for non-technical policy analysts: every chart has a plain-language caption, every metric has a tooltip, and the sidebar filters are labelled in the vocabulary of parliamentary analysis rather than data science.

6.1 Architecture

The app follows a strict top-to-bottom execution model (Streamlit's standard pattern):

- Configuration constants and colour palette are defined at module level.
- `load_data()` reads and preprocesses the CSV. `@st.cache_data` ensures it runs only once per session.
- The sidebar renders all filter widgets and writes to `st.session_state` variables.
- The main panel applies filters to produce the 'filtered' DataFrame.
- Five analytical sections render sequentially: Overview, Framing Intensity, Securitization Signal, Actors, Search & Context.
- Two additional sections: AI Summarization (Section 6) and Database Schema (Section 7).

6.2 Dashboard Sections

Section	Charts / Components	Key Insight Shown
1 · Overview	Stacked area chart (framing over time), donut chart (framing mix), KPI metrics, Key Finding panel	When each framing dominates, absolute vs relative trends
2 · Framing Intensity	Grouped bar chart (yearly density scores), scatter plot (security vs economic)	Which framings are most concentrated in language; pure-security vs mixed utterances
3 · Securitization Signal	Dual-axis chart (utterance count + security overlap %)	Whether the share of securitized utterances is rising, not just the count
4 · Actors & Influence	Horizontal bar charts (top speakers, top securitizers, party rates)	Who drives security framing; party-level differences
5 · Search & Context	Text search, keyword highlighting, sortable expander list	Qualitative validation of quantitative findings
6 · AI Analysis	Three-mode Claude API integration	Utterance summarization, framing reports, orientation prediction
7 · Database Schema	Tabbed SQL viewer (DDL + example queries)	Data architecture for production deployment

6.3 Hover Label Fix

A critical UI issue was identified: Plotly's default hover tooltips on a dark background used white box backgrounds with white text, rendering them unreadable. The fix adds explicit hoverlabel configuration to the shared `plotly_hcss()` theme function:

```
def plotly_hcss(fig, *, show_legend_bottom=True):
    fig.update_layout(
        template='plotly_dark',
        paper_bgcolor=HCSS_NAVY,
        plot_bgcolor=HCSS_NAVY,
        hoverlabel=dict(
            bgcolor=HCSS_NAVY, # dark navy background
            bordercolor=HCSS_GOLD, # gold border
            font=dict(color='white', size=13), # readable white text
        ),
    )
```

6.4 AI Summarization Component

The AI component calls the Anthropic Claude API (claude-sonnet-4-20250514) directly from Streamlit using Python's built-in `urllib`, requiring no additional dependencies. Three analytical modes are available:

Mode	System Prompt Focus	Use Case
Summarise Utterance	Extract core argument; identify dominant framing dimensions; cite specific phrases; note geopolitical actor references	Quick classification check on any utterance from the search panel
Generate Framing Report	Structured HCSS-style analytical memo: Executive Summary Framing Analysis Key Actors Securitization Signal Analytical Note	Policy briefing input; presentation preparation
Predict Political Orientation	Infer party family from language patterns only (no metadata); cite linguistic evidence; state confidence level	Demonstrate ideological distinctiveness of framing styles

API Design Note

The API key is not hardcoded. The Streamlit deployment environment must set the `ANTHROPIC_API_KEY` environment variable. Errors are caught and displayed as informative Streamlit warnings rather than unhandled Python exceptions.

7. Database Schema Design

The schema.sql file in this submission defines a production-ready data warehouse schema optimised for querying parliamentary discourse data at scale.

7.1 Schema Pattern: Star Schema

The schema uses a star schema with one central fact table (fact_utterance) surrounded by dimension tables (dim_date, dim_session, dim_speaker, dim_party). This pattern is standard in analytical data warehouses: join to whichever dimensions you need, group by, aggregate.

Why not a single flat table?

The enriched CSV is effectively a flat table, and it works fine for 9,102 rows. At parliamentary corpus scale, flat tables cause three problems: (1) redundant storage — speaker name and party are repeated on every utterance row; (2) update anomalies — if a speaker's name is corrected, every row must be updated; (3) slow aggregations — without indexes, GROUP BY queries scan the entire table. The star schema solves all three.

7.2 The Bridge Table — Slowly Changing Dimension Type 2

The most architecturally interesting design decision is the bridge_speaker_party table. A naive schema would put party_id directly on dim_speaker, but this cannot represent politicians who change party affiliation mid-corpus. The most prominent example is Pieter Omtzigt, who left the CDA in July 2021.

```
-- SCD Type 2: each row represents a membership interval
CREATE TABLE bridge_speaker_party (
  speaker_id TEXT,
  party_id TEXT,
  valid_from DATE NOT NULL,
  valid_to DATE, -- NULL = currently active / open-ended
  PRIMARY KEY (speaker_id, party_id, valid_from),
  ...
);

-- Correct temporal join: assigns utterance to party on debate date
JOIN bridge_speaker_party bsp
ON s.speaker_id = bsp.speaker_id
AND debate_date BETWEEN bsp.valid_from
AND COALESCE(bsp.valid_to, '9999-12-31')
```

7.3 Pre-Computed Scores vs. On-Demand Calculation

All keyword scores and density values are stored in fact_utterance as pre-computed columns. The pre-computation approach is chosen for three reasons:

- **Performance:** scoring 9,102 rows takes seconds in Python with pre-compiled regex patterns; doing it in SQL for every dashboard query would add latency.

- **Reproducibility:** the scoring logic (lemmatisation, stopword removal, phrase-before-word matching) is complex and would be difficult to replicate faithfully in SQL.
- **Flexibility:** if keyword dictionaries are updated, the ETL pipeline re-runs to regenerate scores; the schema and queries do not change.

7.4 Index Strategy

Index	Column(s)	Covers This Query Pattern
idx_fact_date	date_id	All temporal trend queries (GROUP BY year, month)
idx_date_year	year	Year-level aggregations without joining fact table
idx_date_month	year, month	Monthly trend chart (composite reduces index scans)
idx_fact_speaker	speaker_id	Speaker ranking charts
idx_fact_session	session_id	Session-level aggregations
idx_fact_primary_framing	primary_framing	Sidebar framing filter (WHERE primary_framing IN ...)
idx_fact_energy_security	is_energy_security	Energy × security overlap queries
idx_fact_framing_date	primary_framing, date_id	Most expensive dashboard query: framing trend filtered by date

8. Analytical Findings

8.1 Securitization Trajectory (2019–2022)

The most robust finding from this analysis is a marked, non-linear increase in security-framed energy language in the Dutch parliament, with a clear structural break in early 2022.

Year	Utterances	Avg. Security Density	Avg. Securitization Index	Energy×Security Rate
2019	~2,300	1.92	0.41	~14%
2020	~2,100	2.10	0.44	~15%
2021	~2,200	2.37	0.50	~16%
2022	~2,500	6.94	1.21	~21%

The 227.3% increase in security density from 2021 to 2022 is statistically significant by Mann-Whitney U test. The securitization index crossing 1.0 in 2022 is analytically meaningful: it means that in 2022, security language in energy debates outweighed economic + climate + minerals language combined, a qualitative shift, not just a quantitative one.

8.2 Frame Co-occurrence

Among energy-relevant utterances (energy_score > 0), the co-activation rates reveal which framings travel together:

Frame Pair	Co-occurrence Rate	Interpretation
Energy + Economic	32.0%	Very common: energy affordability debates naturally blend technical and economic language.
Energy + Climate	30.5%	Common: climate and energy are treated as linked policy domains throughout the period.
Energy + Security	17.2%	Rising sharply in 2022: the securitization signal this project tracks.
Energy + Minerals	2.3%	Rare: critical materials are not yet a mainstream parliamentary framing, despite their strategic importance.

8.3 Actor-Level Patterns

Speaker-level analysis reveals two distinct profiles in energy-security debates:

- **High-volume speakers** (Mark Rutte, Rob Jetten, Wopke Hoekstra, Sigrid Kaag) appear frequently because they held ministerial positions during the period, not necessarily because they securitize more intensely.
- **High-intensity securitizers** (Ruben Brekelmans, Sven Koopmans, Sjoerd Sjoerdsma — all from VVD and D66 foreign affairs and defence committees) achieve high average security density with fewer total utterances, suggesting specialised expertise rather than general volume.
- The Eerste Kamer (Senate) shows lower volume but comparable security density to the Tweede Kamer, suggesting it engages with the same securitization discourse in a more focused, less voluminous way.

8.4 The Critical Materials Gap

Strategic Finding

Critical minerals (lithium, cobalt, rare earths, semiconductors) account for only 2.3% of primary framing and 2.3% of energy-utterance co-occurrence across the entire 2019–2022 period. This is analytically significant: at the same time that Europe was becoming acutely aware of semiconductor supply chain dependencies (illustrated by the 2020–2022 chip shortage) and mineral import dependencies from China, Dutch parliamentary discourse had not yet caught up. This gap between strategic reality and parliamentary attention is itself a finding worth highlighting to HCSS researchers.

9. Technology Stack

Component	Technology	Version	Rationale
Data ingestion	Python / re / csv	3.11+	Standard library: no dependency overhead; regex fast for pattern matching at scale
Text processing	NLTK (WordNetLemmatizer)	3.8+	Mature NLP library; WordNet lemmatiser handles parliamentary text well
XML parsing	lxml + XPath	4.x	Fastest Python XML parser; native TEI namespace support
Data manipulation	Pandas	2.0+	Industry standard; handles 9k-row CSV operations in milliseconds
Statistical analysis	SciPy + scikit-learn	1.x / 1.4+	Mann-Whitney U, chi-square from SciPy; K-Means, PCA from scikit-learn
Dashboard framework	Streamlit	1.32+	Rapid prototyping; Python-native; no JavaScript required
Visualisation	Plotly Express / Graph Objects	5.18+	Interactive charts; dark theme; dual-axis support
AI integration	Anthropic Claude API	claude-sonnet-4	State-of-the-art language model; structured system prompts for consistent output

10. Limitations & Future Work

10.1 Current Limitations

- Keyword-based scoring is a proxy for framing, not a direct measure. A speaker can use security language ironically or critically without actually securitizing energy. Sentiment-aware scoring or manual annotation would improve precision.
- Party metadata is partially incomplete due to TEI extraction issues. The name-based fallback covers the most active speakers but leaves the majority of utterances unmatched.
- The corpus covers only 2019–2022. The post-invasion period (late 2022 onwards) would likely show even higher securitization and potentially the emergence of critical minerals as a mainstream frame.
- English translations (used here) may lose rhetorical nuance present in the original Dutch. Future work should validate findings against the Dutch-language corpus.
- K-Means clustering requires specifying *k* in advance. The silhouette scoring helps select *k*, but agglomerative clustering with dendrogram inspection would be more robust.

10.2 Future Improvements

- **LLM-assisted keyword expansion:** prompt a language model to suggest additional security framing terms given examples from the corpus, then validate with `keyword_audit.py`.
- **Sentence-level scoring:** the current system scores entire utterances. Sentence-level scoring would enable detection of within-speech framing shifts.
- **Full TEI party extraction:** fix the root corpus file parsing in `enrich_speakers.py` to correctly extract party affiliation for all 337 speakers.
- **Sentiment layer:** integrate a pre-trained Dutch-language sentiment model (e.g. RobBERT) to distinguish securitizing from critical or ironic security language references.
- **Topic modelling:** apply BERTopic or LDA to the normalised text corpus to discover emergent themes without relying on pre-defined keyword dictionaries.
- **Database implementation:** load the enriched CSV into a SQLite or DuckDB database using the `schema.sql` schema, enabling sub-millisecond aggregation queries for real-time dashboard filtering.

11. Setup & Usage

11.1 Installation

1. Install Python dependencies

```
pip install -r requirements.txt
```

requirements.txt contains:

```
# streamlit>=1.32.0
```

```
# pandas>=2.0.0
```

```
# plotly>=5.18.0
```

```
# nltk>=3.8.1
```

```
# lxml>=4.9
```

```
# scikit-learn>=1.4
```

```
# scipy>=1.12
```

2. Download NLTK data (first run only)

```
python -c "import nltk; nltk.download('wordnet'); nltk.download('stopwords'); nltk.download('punkt')"
```

11.2 Pipeline Execution

Stage 1: Process raw ParlaMint text files

```
python process_parlamint.py \  
  --data_dir ./ParlaMint-NL-en.txt \  
  --out_dir ./processed \  
  --years 2019 2020 2021 2022
```

Stage 2: Enrich with speaker metadata from TEI/XML

```
python enrich_speakers.py \  
  --processed_csv ./processed/energy_security_utterances.csv \  
  --tei_dir ./ParlaMint-NL-en.TEI.ana \  
  --out_csv ./processed/energy_security_utterances_enriched.csv
```

Stage 3 (optional): Audit keyword coverage

```
python keyword_audit.py \  
  --processed_csv ./processed/energy_security_utterances.csv \  
  --out_dir ./keyword_audit
```

Stage 4 (optional): Run advanced statistical analysis

```
python advanced_analysis_energy_security.py \  
  --input_csv ./processed/energy_security_utterances_enriched.csv \  
  --out_dir ./analysis_outputs
```

Stage 5: Launch the dashboard

```
export ANTHROPIC_API_KEY=your_key_here # required for AI features  
streamlit run app.py
```

11.3 File Structure

```
project/  
├─ app.py # Streamlit dashboard  
├─ process_parlamint.py # Stage 1: ingestion & scoring  
├─ enrich_speakers.py # Stage 2: speaker metadata  
├─ advanced_analysis_energy_security.py # Stage 4: statistics  
├─ requirements.txt # Python dependencies  
├─ schema.sql # Data warehouse SQL schema  
├─ processed/  
│   ├─ energy_security_utterances.csv  
│   └─ energy_security_utterances_enriched.csv  
└─ analysis_outputs/ # Stage 4 outputs (22 files)
```